
Your Language Model is Its Own Critic: Reinforcement Learning with Value Estimation from Actor’s Internal States

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Reinforcement learning with verifiable rewards (RLVR) for Large Reasoning Mod-
2 els hinges on baseline estimation for variance reduction, but existing approaches
3 pay a heavy price: PPO requires a policy-model scale critic, while GRPO needs
4 multiple rollouts per prompt to keep its empirical group mean stable. We intro-
5 duce **POISE** (Policy Optimization with Internal State Value Estimation), which
6 obtains a baseline at negligible cost by using the policy model’s internal signals
7 already computed during the policy forward pass. A lightweight probe predicts
8 the expected verifiable reward from the hidden states of the prompt and generated
9 trajectory, as well as token-entropy statistics, and is trained online alongside the
10 policy. To preserve gradient unbiasedness despite using trajectory-conditioned
11 features, we introduce a cross-rollout construction that predicts each rollout’s value
12 from an independent rollout’s internal states. Because POISE estimates prompt
13 value using only a single rollout, it enables higher prompt diversity for a fixed
14 compute budget during training. This reduces gradient variance for more stable
15 learning and also eliminates the compute overhead of sampling costs for detecting
16 zero-advantage prompts. On Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B
17 across math reasoning benchmarks, POISE matches DAPO while requiring less
18 compute. Moreover, its value estimator shows similar performance to a separate
19 LLM-scale value model and generalizes to various verifiable tasks. By leveraging
20 the model’s own internal representations, POISE enables more stable and efficient
21 policy optimization.¹

22 1 Introduction

23 Large language models (LLMs) have recently shown remarkable improvements on complex reasoning
24 tasks by generating long chains of thought before committing to a final answer [12, 36]. A central
25 driver of this progress has been reinforcement learning with verifiable rewards (RLVR), which
26 optimizes the model using outcome-level rewards [9, 14]. To reduce reward variance and the resulting
27 training instability, a baseline is subtracted from the reward to form an advantage—a measure of
28 how much better a given response is relative to what the model would typically achieve. Obtaining a
29 reliable baseline is therefore central to stable and efficient RLVR.

30 Yet existing approaches pay a significant computational price to do so. Proximal Policy Optimization
31 (PPO) [24] trains an LLM-scale critic with the policy to produce per-token baseline values; critic
32 must process the full generated sequence at every update, roughly doubling memory consumption and
33 increasing the optimization complexity. Group Relative Policy Optimization (GRPO) [25] sidesteps

¹We will release the code upon the publication of the paper.

the critic by estimating a per-prompt baseline as the mean reward over a group of sampled rollouts, but this trades parameters for samples: a reliable estimation of the baseline requires multiple rollouts per prompt, which under a fixed compute budget reduces in-batch prompt diversity and, in turn, inflates the variance of gradient estimates [7] (see § 2.3). Substantial compute is also spent on uninformative prompts, for which all rollouts receive identical rewards and therefore yield zero advantage [37]. As reasoning trajectories grow longer, both costs compound and consume compute that could otherwise be used for learning. Underlying both approaches is the same bottleneck: producing a baseline demands extra resources. This motivates the central question of our work: *Can an effective baseline be extracted from the computations already performed during policy training?*

We suggest that a promising answer to this question is to leverage the information encoded in the policy model’s own internal representations to estimate the baseline. This hypothesis is grounded in a growing body of work showing that hidden states of LLMs and LRMs encode outcome-relevant information such as perceived difficulty, capability boundaries, and answer correctness, which can serve as a highly informative proxy for expected rewards. Yet these signals have been treated purely as diagnostic tools at inference time, leaving their potential to inform training entirely unexplored.

In this paper, we propose **POISE** (Policy Optimization with Internal State Value Estimation), a reinforcement learning algorithm that turns the model’s internal states into a value model. Concretely, we train a lightweight probe that predicts the value $V^\pi(x) = \mathbb{E}_{y \sim \pi(\cdot|x)}[R(x, y)]$ from internal signals collected at two levels. The first is a *prompt-level* feature, extracted from hidden states at the final prompt tokens before generation begins, which captures how the model represents the prompt and its anticipated difficulty under the current policy. The second is a *trajectory-level* feature, comprising hidden states taken when the model’s reasoning ends together with token-level entropy. Because using rollout-dependent signals in the baseline biases the gradient estimator, we pair each rollouts with a second, independent rollout from the same prompt. The probe predicts the *paired* rollout’s value thereby making the value independent to the corresponding rollout. This cross-rollout architecture keeps the baseline conditionally independent of the action which otherwise introduce bias into the gradient estimator [29, 33], so the probe is driven to recover the policy’s expected reward $V^\pi(x)$ rather than to memorize trajectory-specific outcomes. Trained jointly with the policy on a sliding buffer of recent rollouts, our value estimator tracks the evolving policy with negligible overhead.

Our method offers several concrete advantages over existing approaches. Unlike PPO, the baseline is supplied by a lightweight value estimator rather than an LLM-scale critic. Compared to GRPO, our method requires only a pair of rollouts rather than a large group; the saved budget can be redirected to more distinct prompts per batch, improving training stability. Moreover, because the value estimator provides a lightweight continuous baseline for each rollout, POISE avoids the extra sampling needed to identify and discard degenerate zero-advantage prompt groups.

We validate these claims experimentally. POISE matches DAPO [37]: a state of the art, GRPO-based RL algorithm in LLM reasoning, with less compute. We also show that our lightweight value estimator performs similar to an LLM-scale value model in performance (Figure 1), despite relying only on signals already produced during the policy’s forward pass. Beyond these performance results, we analyze the estimator itself (§ 5), identifying which layers and signals contribute most to value prediction and tracking how the estimator evolves alongside the policy during training. Finally, we demonstrate that the estimator generalizes beyond mathematical reasoning, yielding consistent gains on coding, tool-calling, and instruction-following tasks.

Overall, we show that internal representations of reasoning models can move beyond their conventional use as diagnostic tools for reasoning behavior and serve as practical optimization signals for reinforcement learning. Without group-relative baselines or a separate critic model, our method provides a compute-efficient path toward stable and scalable RLVR for large reasoning models.

2 Preliminaries

2.1 Policy Gradient and Baseline Estimation

We formulate RLVR for LLM reasoning as a contextual bandit problem over prompt–response pairs [31]. Given a prompt $x \sim \mathcal{D}$ and a response $y \sim \pi_\theta(\cdot | x)$ sampled from the policy model, the objective is to maximize the expected verifiable reward $R(x, y)$,

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [R(x, y)]. \quad (1)$$

86 By the policy gradient theorem,

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} [R(x, y) \nabla_{\theta} \log \pi_{\theta}(y | x)], \quad (2)$$

87 which yields the REINFORCE estimator [33]. In practice, this estimator is typically combined with
88 a baseline $b(x)$ to reduce variance [28], giving the advantage $A(x, y) = R(x, y) - b(x)$ and the
89 gradient estimator

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} [(R(x, y) - b(x)) \nabla_{\theta} \log \pi_{\theta}(y | x)]. \quad (3)$$

90 The standard near-optimal choice for variance reduction is the value function [8, 32]

$$V^{\pi_{\theta}}(x) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [R(x, y)], \quad (4)$$

91 which is unknown and must be estimated in practice. PPO approximates $V^{\pi_{\theta}}(x)$ with a learned critic
92 v_{ϕ} that is trained jointly with the policy, providing a direct parametric estimate of the value function.
93 GRPO instead samples a group of G responses $\{y^{(1)}, \dots, y^{(G)}\}$ for the same prompt and uses their
94 mean reward as an empirical prompt-level baseline,

$$b_{\text{GRPO}}(x) = \frac{1}{G} \sum_{j=1}^G R(x, y^{(j)}), \quad (5)$$

95 obtaining the baseline directly from on-policy rollouts.

96 2.2 Unbiasedness Condition for Baselines

97 Subtracting a baseline preserves the unbiasedness of the policy gradient only when the baseline term
98 has zero expectation:

$$\mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [b(x) \nabla_{\theta} \log \pi_{\theta}(y | x)] = 0. \quad (6)$$

99 This condition holds when the baseline is conditionally independent of the sampled response y given
100 the prompt x , in which case

$$\mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [b(x) \nabla_{\theta} \log \pi_{\theta}(y | x)] = b(x) \nabla_{\theta} \sum_y \pi_{\theta}(y | x) = 0. \quad (7)$$

101 Equivalently, a baseline may depend only on the prompt or on any quantity that is independent of the
102 sampled response given the prompt. Violating this condition biases the gradient and can drive the
103 policy to converge suboptimally. We therefore adopt a *cross-rollout* construction, where the baseline
104 for a response is computed from another independent response, preserving Eq. (6).

105 2.3 Gradient Variance and Number of Prompts in the Batch

106 A baseline estimator that requires fewer rollouts per prompt can reallocate the same completion
107 budget toward more distinct prompts in each batch. This section formalizes why such prompt diversity
108 matters for policy optimization. We show that, under a fixed compute budget, allocating rollouts
109 across more distinct prompts reduces the noise of the gradient estimate.

110 Let B be the total number of completions in a training batch, with n distinct prompts and m
111 completions each, so $B = n \cdot m$. For prompt $x^{(i)} \sim D$ and completion $y^{(ij)} \sim \pi_{\theta}(\cdot | x^{(i)})$, define
112 the per-sample gradient:

$$Z(x, y) = \nabla_{\theta} \log \pi_{\theta}(y | x) (R(x, y) - b(x)) \quad (8)$$

113 where $b(x)$ is a baseline. The batch gradient estimator is:

$$\hat{g} = \frac{1}{n} \sum_{i=1}^n \frac{1}{m} \sum_{j=1}^m Z(x^{(i)}, y^{(ij)}). \quad (9)$$

114 **Proposition 1** (Gradient variance decomposition). *Let Σ_w and Σ_b denote the within-prompt and*
115 *between-prompt covariance matrices of Z . Both Σ_w and Σ_b are fixed properties of (D, π_{θ}, R) ,*
116 *independent of the allocation (n, m) . Then:*

$$\text{Cov}(\hat{g}) = \frac{1}{B} \Sigma_w + \frac{m}{B} \Sigma_b. \quad (10)$$

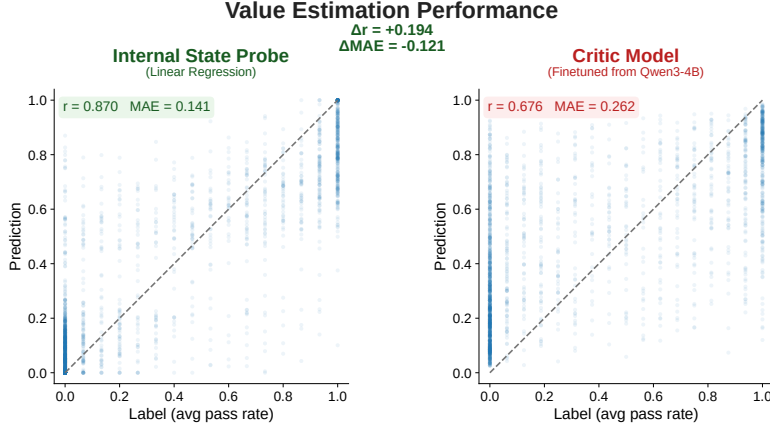


Figure 1: Comparing value prediction between our internal state probe and a separately trained critic model. Predictions are compared against empirical Avg@8 scores. Our probe achieves higher Pearson correlation (r) and lower MAE, indicating that the policy’s own internal representations provide an effective low-cost signal for value estimation.

117 *Proof.* See § B.1. □

118 **Corollary 1** (Optimal allocation). *For a fixed budget B and baseline $b(x)$, the variance of $\hat{g}$ is*
 119 *monotonically non-decreasing in m (in the Loewner order) and is minimized at $m = 1$, $n = B$.*

120 *Proof.* See § B.2. □

121 In other words, given the same total budget, using as many diverse prompts as possible is critical to
 122 stable learning (i.e., $m = 1$ or 2). Yet GRPO requires repeated sampling from the same prompt to
 123 estimate a faithful baseline $b(x)$. This motivates our method of estimating a reliable baseline with
 124 minimal sampling without training a separate value network as in PPO.

125 We provide an extended discussion of related work on RLVR value estimation and outcome-relevant
 126 internal representations in § A.

127 3 Policy Optimization with Internal State Value Estimation (POISE)

128 We now introduce POISE, which leverages the policy model’s internal state signals for value estima-
 129 tion in RLVR. We first show that a lightweight probe can predict the value function, i.e., the expected
 130 verifier reward, directly from the policy model’s internal states (§ 3.1). We then integrate this probe
 131 into policy optimization to compute per-rollout advantages, yielding the full POISE algorithm without
 132 requiring a separate LLM-scale value model (§. 3.2).

133 3.1 Value Function Estimation from Policy Model Internal States

134 We introduce a probe designed to estimate baseline values directly from the policy model’s internal
 135 representations. Since the viability of this method hinges on the presence of such information, we
 136 additionally present preliminary empirical results demonstrating that these internal states inherently
 137 encode the necessary signals for accurate value estimation.

138 **Probe prediction objective.** The probe is trained to predict the prompt-level value under the current
 139 policy, defined as the expected verifier reward:

$$V^{\pi_\theta}(x) = \mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[R(x, y)].$$

140 Since the ground-truth quantity is unknown, we instead sample K rollouts for each prompt x ,
 141 $y^{(1)}, \dots, y^{(K)} \sim \pi_\theta(\cdot|x)$, and collect their verifier rewards $r^{(i)} = R(x, y^{(i)}) \in \{0, 1\}$. For the

supervised example associated with rollout $y^{(i)}$, we use the leave-one-out Monte Carlo target as its gold value:

$$\widehat{V}_{-i}(x) = \frac{1}{K-1} \sum_{j \neq i} r^{(j)}.$$

By excluding $r^{(i)}$, $\widehat{V}_{-i}(x)$ remains conditionally independent of the input rollout $y^{(i)}$ given x , while still estimating $V^{\pi_\theta}(x)$ in expectation. This prevents the target from leaking the reward of the same rollout whose features are used by the probe.

Probe input features. As shown in Figure 2 (left), each supervised example for our probe is indexed by a prompt and one rollout, $(x, y^{(i)})$. For each pair, we construct the probe input from three complementary signals produced during the forward pass of the current policy π_θ . (All hidden-state features are extracted from a fixed layer ℓ , which we omit below for readability.) Let $H_{\theta,t}^{(i)}$ denote the residual-stream hidden state at token position t for $(x, y^{(i)})$, and let P and $R^{(i)}$ denote the final n prompt-token and reasoning-token positions, respectively.

First, we use the prompt-state feature $h_{\theta,p}^{(i)} = \text{Avg}_{t \in P} H_{\theta,t}^{(i)}$, motivated by evidence that prompt hidden states encode pre-generation estimates of difficulty and capability boundaries [42]. Second, we use the reasoning-state feature $h_{\theta,r}^{(i)} = \text{Avg}_{t \in R^{(i)}} H_{\theta,t}^{(i)}$, since trajectory-level hidden states can expose value-relevant information not available from the prompt states alone [39]. Third, we use token-level entropy statistics $u_\theta^{(i)}$ as lightweight uncertainty features [30]. The final probe input is

$$\phi_\theta^{(i)} = [h_{\theta,p}^{(i)}; h_{\theta,r}^{(i)}; u_\theta^{(i)}].$$

We ablate these input components in § 5 and the hidden-state extraction hyperparameters in § F.1. It is important to clarify that, while the input features include the generated reasoning, the estimator learns to predict the prompt-based value, rather than verifying its own reasoning, because the prediction target during training is the expected reward derived from other responses.

Probe implementation. We train lightweight regressors g_f to minimize the following loss.

$$\mathcal{L}_{\text{value}}(f) = \mathbb{E}_{x,i} \left[\left(g_f(\phi_\theta^{(i)}) - \widehat{V}_{-i}(x) \right)^2 \right].$$

Although our framework can theoretically support any regression architecture, we implement the probe using linear regression because its computational efficiency allows for fast, lightweight updates at each training step. We provide an ablation of probe designs in § F.2, and provide detailed implementations and hyperparameters in § C.3.

Preliminary experiment. Before using the probe for policy optimization, we first test whether the policy model’s internal states contain enough information to reliably estimate the prompt-level value. We construct a held-out value-prediction benchmark from reward-labeled rollouts of the DAPO-Math [37] dataset and compare two estimators trained on the same data: (1) a separate policy-scale critic model as a strong baseline (see § E.1 for details), and (2) our lightweight probe over the policy model’s internal state and entropy features. We evaluate both estimators on held-out prompts by comparing their predictions against the empirical Avg@8 reward.

Figure 1 shows that probes over the policy’s internal states achieve better held-out value prediction than the separate value model, despite adding only a lightweight regression head. This shows that the policy model’s own activations encode a compact signal about prompt difficulty and policy-specific uncertainty, which can be leveraged for value estimation at negligible cost.

3.2 Policy Optimization with Cross-Rollout Baselines

We now integrate the internal state probe into RL training as a value estimator, forming the full POISE algorithm (Figure 2 right).

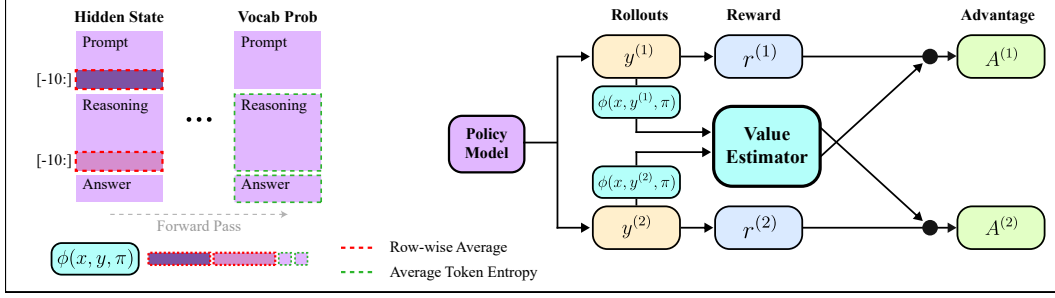


Figure 2: Overview of POISE. **Left:** Probe features $\phi(x, y, \pi)$ combine hidden states with token entropy. **Right:** The value estimator predicts each rollout’s baseline from the other rollout’s features.

Two rollouts per prompt. For each prompt $x \sim \mathcal{D}$ in the training batch we sample two independent rollouts from the current policy,

$$y^{(1)}, y^{(2)} \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta_{\text{old}}}(\cdot | x), \quad (11)$$

and evaluate their verifiable rewards $R(x, y^{(1)})$ and $R(x, y^{(2)})$.

Cross-rollout baseline and advantage. The baseline for each rollout is predicted from the internal signals of the *other* rollout:

$$b^{(1)}(x) = g_f(\phi^{(2)}), \quad b^{(2)}(x) = g_f(\phi^{(1)}), \quad (12)$$

This yields the cross-rollout advantages

$$A^{(i)}(x) = R(x, y^{(i)}) - b^{(i)}(x), \quad i \in \{1, 2\}. \quad (13)$$

By construction, the baseline used to update $y^{(i)}$ depends only on the independently sampled rollout $y^{(j)}$, $j \neq i$, satisfying the conditional-independence condition in Eq. (6).

PPO-style policy update. We optimize the policy with a PPO-style clipped surrogate objective. Let $r_t^{(i)}(\theta) = \pi_{\theta}(y_t^{(i)} | x, y_{<t}^{(i)}) / \pi_{\theta_{\text{old}}}(y_t^{(i)} | x, y_{<t}^{(i)})$ be the importance ratio at token t of rollout i . The objective is

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y^{(1)}, y^{(2)} \sim \pi_{\theta}(\cdot | x)} \left[\frac{1}{2} \sum_{i=1}^2 \frac{1}{|y^{(i)}|} \sum_{t=1}^{|y^{(i)}|} \min \left\{ r_t^{(i)}(\theta) A^{(i)}(x), \right. \right. \\ \left. \left. \text{clip} \left(r_t^{(i)}(\theta), 1 - \epsilon, 1 + \epsilon \right) A^{(i)}(x) \right\} \right]. \quad (14)$$

which we maximize with respect to θ over multiple inner epochs per batch.

Online estimator training with a trajectory buffer. The value estimator g_f is trained jointly with the policy on a sliding buffer of recent trajectories. At each step, for each prompt x with two independent rollouts $(y^{(1)}, y^{(2)})$, we construct value-estimator examples $\{(x, \phi(x, y^{(i)}), \hat{V}_{-i}(x))\}_{i=1,2}$, where $\hat{V}_{-i}(x) = R(x, y^{(j)})$, $j \neq i$. We update f by minimizing a regression loss over the union of these newly generated examples and a buffer of examples from the most recent n steps. The buffer stabilizes the training signal under policy drift, while the joint update keeps g_f aligned with the value function of the evolving policy. Because g_f is a lightweight probe over signals already computed during the forward pass, this update is negligible in cost. The full procedure is summarized in Algorithm 1.

4 Experiments

4.1 Experimental Setup

Training. We instantiate our method on Qwen3-4B [35] and DeepSeek-R1-Distill-Qwen-1.5B [9], training on the English subset of DAPO-Math-17K [37] with batch sizes of 1024 and 512

Table 1: Performance comparison on olympiad level mathematical reasoning benchmarks. We report Avg@32 accuracy across various datasets. Our proposed internal state value estimation method (POISE) achieves competitive performance with the baseline models.

Model	Method	Benchmark							Avg
		AMC23	AMC24	AIME24	AIME25	AIME26	HMMT25	BRUMO25	
Qwen3-4B	<i>base</i>	0.422	0.319	0.263	0.196	0.244	0.129	0.217	0.258
	DAPO	0.876	0.607	0.490	0.475	0.457	0.267	0.384	0.508
	POISE (Ours)	0.891	0.592	0.469	0.437	0.443	0.280	0.387	0.500
Deepseek-Distill-Qwen-1.5B	<i>base</i>	0.169	0.078	0.067	0.067	0.104	0.021	0.042	0.078
	DAPO	0.697	0.447	0.254	0.219	0.198	0.065	0.191	0.296
	POISE (Ours)	0.694	0.446	0.270	0.234	0.213	0.066	0.198	0.303

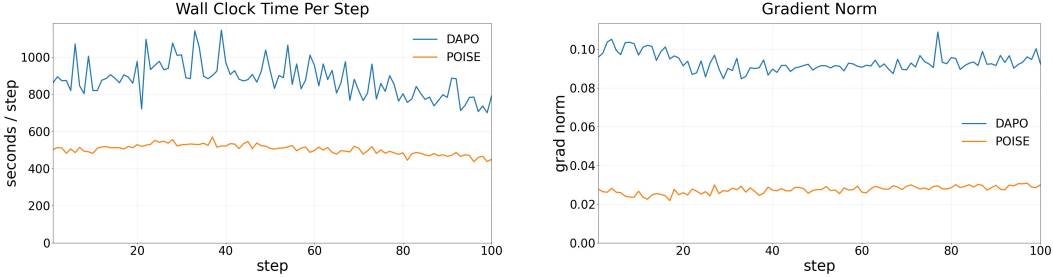


Figure 3: Comparison of training dynamics between POISE and DAPO on Deepseek-Distill-Qwen-1.5B. **Left:** wall-clock time per training step. **Right:** gradient norm at each step.

on B200 GPUs. Rollouts are sampled with temperature 1.0 and top-p 1.0. Our main baseline is DAPO [37]: a state of the art, GRPO-based RLVR algorithm for mathematical reasoning. We adopt the implementation of Zheng et al. [41], which improves the efficiency of DAPO’s dynamic sampling. Full hyperparameters are provided in § C.4.

Evaluation. We evaluate our method on a suite of olympiad mathematical reasoning benchmarks: AMC23/24 [18], AIME24/25/26 [19], HMMT25 [10], and BRUMO25 [2]. For each benchmark, we report Avg@32, using temperature 0.6 and top-p 0.95 following common reasoning-model evaluation settings [9, 35]. By averaging over 32 sampled responses per problem, this protocol provides a reliable estimate of each model’s expected reasoning performance. We also compare training efficiency by analyzing the wall-clock time each method requires to achieve comparable reasoning performance. Detailed descriptions of each dataset, the full evaluation protocol are provided in § C.5.

4.2 Main Results on Math Reasoning Benchmarks

Table 1 reports the main results on olympiad-level mathematical reasoning benchmarks. For Qwen3-4B, POISE achieves an average Avg@32 score of 0.500, which is close to DAPO’s 0.508, while outperforming DAPO on AMC23, HMMT25, and BRUMO25. For Deepseek-Distill-Qwen-1.5B, POISE improves the average Avg@32 score from 0.296 to 0.303 over DAPO, with gains on AIME24, AIME25, AIME26, HMMT25, and BRUMO25. Across both model scales, these results indicate that POISE achieves performance comparable to a state-of-the-art RL algorithm while replacing group-relative baseline estimation with lightweight internal state value estimation. Detailed training dynamics are provided in D.

Training efficiency and stability. Figure 3 (left) shows that POISE requires substantially less wall-clock time per step than DAPO. The difference comes from how the two methods obtain usable advantage signals. In DAPO, the group-mean baseline becomes uninformative when all rollouts for a prompt receive the same reward, so dynamic sampling must first generate a full group of N rollouts to check whether the prompt yields nonzero advantages. When a group is degenerate, its rollouts are excluded from the final training batch for each step, forcing DAPO to sample additional groups until enough effective examples are collected. In contrast, POISE predicts the expected verifier reward as a

continuous value from internal state signals already produced during generation, thereby avoiding degeneration and saving a substantial amount of rollout compute. Concretely, in our setting, on DeepSeek-R1-Distill-Qwen-1.5B, reaching the same performance level takes roughly 24 hours of wall-clock time with DAPO on a single B200 GPU, compared to about 18 hours with POISE. We observe a similar trend on Qwen3-4B: POISE requires about 36 hours on two B200 GPUs, whereas DAPO takes 49 hours under the same hardware setting.

We further examine whether POISE leads to more stable optimization. DAPO and our method form the same gradient estimator through Eq. (9) and differ in how the baseline $b(x)$ is constructed. The expected squared norm of a policy-gradient estimator decomposes into the true gradient signal and an estimation-noise term. Since the true gradient depends only on the current policy and data, methods at similar training progress should have comparable signal magnitude; differences in gradient norm therefore mainly reflect differences in estimator noise. Under the same batch budget, POISE fits more distinct prompts than DAPO, which, in principle, reduces gradient variance according to § 2.3 and stabilizes training. Figure 3 (right) confirms this empirically. Our gradient-norm stays consistently lower than DAPO’s throughout training.

Training dynamics of value estimator. To evaluate whether the value estimator reliably tracks the evolving policy, we compute the online MAE (mean absolute error) between its predicted baseline values and the empirical mean reward of rollouts sampled from the current policy (Figure 4). The online MAE stays relatively stable across training, indicating that the estimator remains calibrated to the rewards produced by the current policy. Meanwhile, the variance reduction ratio remains around 30% after the initial phase, showing that the learned baseline reduces the reward variance by roughly one third when forming the advantage. Together, these results suggest that the online-trained estimator adapts to policy changes and provides a stable baseline throughout training.

5 Analysis of the Value Estimator

Comparison to an online policy-model scale critic. The previous training dynamics analysis evaluates whether our estimator serves as a stable baseline during policy optimization. Here, we additionally compare with a separately trained policy-model scale critic under policy drift. Using the Qwen3-4B training log from our main experiments, we train a critic model just like in § 3.1, but on accumulated reward-labeled rollouts and evaluate both estimators every 10 steps against empirical Avg@8 values from the corresponding actor checkpoint. As shown in Figures 7 and 8, the critic is slightly more accurate, likely due to its larger capacity and continual training on the expanding rollout log. Nevertheless, our estimator closely tracks the critic while requiring only a lightweight probe over features already produced by the policy forward pass.

Generalizability across domains and models. Next, we evaluate the estimator’s generalizability across multiple RLVR domains and policy models. For datasets, we include two mathematical reasoning tasks from DAPO-Math 17K [37] and DeepScaleR [17], coding tasks from AceCoder [38], tool-calling dialogues from ToolDial [26], and instruction-following tasks from IF-RLVR [22]. For policy models, we consider Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B/7B. For each domain-model pair, we train both our estimator and a critic model using the same data, and compare their predictions against the actual avg@8 scores of the target policy model (For detailed settings, refer to § E.3). We report representative results in Table 2 and the full results in Table 7. The estimator is competitive with and often more accurate than the critic model, which suggests that the policy’s hidden states expose a useful signal about whether the model is likely to produce a verifiably correct response. We therefore view our main experiments on math domain (§4.2) as a proof of concept for a more general RLVR mechanism: whenever verifiable feedback is available, a lightweight estimator can be trained online from the policy’s own internal states, providing a cheap value estimate without an auxiliary critic or large rollout groups.

Ablations. We ablate our value estimator along three axes: input features, hyperparameters for hidden state extraction, and probe architecture. We first ablate the input features used by our estimator. We use the same settings as the experiment in § 3.1 and train a value estimator with the following features. First, we evaluate the three core features of our method—prompt hidden states, reasoning hidden states, and vocabulary entropy. Following a prior work [5], we also evaluate response length. As shown in Table 3, trajectory-level features such as reasoning hidden states and mean entropy

Table 2: Performance of our estimator across multiple domains (Qwen3-4B). We compare against a separately trained critic and report MAE and Pearson correlation r . Full results are in Table 7.

Domain	Dataset	Critic		Ours	
		MAE ↓	r ↑	MAE ↓	r ↑
Math	DAPO-Math	0.262	0.676	0.141	0.870
	DeepScaleR	0.393	0.384	0.231	0.609
Coding	AceCoder	0.499	0.056	0.234	0.612
Tool	ToolDial	0.303	0.440	0.188	0.840
Instruction	IF-RLVR	0.350	0.150	0.195	0.642

Table 3: Ablation of estimator input features (Qwen3-4B). We report MAE and Pearson correlation r after training the estimator with only one feature type.

Input feature	MAE ↓	r ↑
only prompt hidden states	0.234	0.569
only reasoning hidden states	0.132	0.821
only mean entropy	0.152	0.780
only response length	0.251	0.494
Full estimator	0.126	0.838

are influential in our value estimator’s performance: using either of the two retains much of the estimator’s accuracy. On the other hand, prompt hidden states show relatively low performance, and ablating the response length provides little or no improvement.

Next, we compare hyperparameter values used during hidden state extraction, such as layer index and mean pooling token size. As detailed in § F.1, our results show that mid-later layers are optimal, and the performance of the probe is not sensitive to token length.

Lastly, we compare our linear probe design with heavier models, such as Multi-Layer Perceptrons (MLPs). The results (Table 12) indicate that linear regression is just as effective as—and sometimes surpasses—the MLP probes. We attribute this to findings from prior work, which demonstrate that many semantic features are encoded as linear directions within the Transformer’s internal representations [6, 21]. Consequently, a linear probe is naturally well-suited to extract these value signals efficiently without the need for additional model complexity or the risk of overfitting.

6 Conclusion

We introduce **POISE** (Policy Optimization with Internal State Value Estimation), which predicts rollout value from the policy’s internal states instead of relying on a group-mean baseline or a separate critic. To preserve unbiasedness, we couple this baseline with a cross-rollout construction. Our method achieves performance comparable to DAPO on mathematical reasoning benchmarks at a lower computational cost and more stable training. Finally, we show that our value estimator performs as well as a separate policy-scale value model and can generalize to other verifiable tasks.

7 Limitations and Future Work

Our experiments are conducted under a fixed compute budget, and while the trends we report are consistent across backbones and benchmarks, characterizing the behavior of internal state value estimation under substantially longer training horizons remains an interesting direction that we leave to future work with greater compute resources.

Several extensions naturally follow from our framework. First, our current estimator predicts value at the sequence level; extending it to token-level credit assignment would yield finer-grained advantages that more precisely reward the tokens responsible for a successful trajectory and penalize those that derail it, an effect known to be especially impactful for long reasoning trajectories [16, 30]. Second, internal state value estimates may also be useful beyond policy gradient optimization: for preference learning algorithms such as Direct Preference Optimization [23], they could inform the construction of response pairs by identifying rollouts with meaningfully different predicted values, potentially yielding more informative preference comparisons. Third, although we focused on mathematical reasoning as a controlled testbed, applying it to RL training for agentic reasoning and instruction-following tasks is a promising next step toward broader RLVR deployment.

References

- [1] Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 967–976, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.68. URL <https://aclanthology.org/2023.findings-emnlp.68/>.
- [2] Brown University Math Olympiad Team. BrUMO 2025: Brown University Mathematics Olympiad. <https://www.brumo.org>, 2025. Inaugural competition, held April 4–5, 2025, Brown University, Providence, RI.
- [3] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=ETKGuby0hcs>.
- [4] Iván Vicente Moreno Cencerrado, Arnau Padrés Masdemont, Anton Gonzalvez Hawthorne, David Demitri Africa, and Lorenzo Pacchiardi. No answer needed: Predicting LLM answer accuracy from question-only linear probes, 2026. URL <https://openreview.net/forum?id=0hN25uxVab>.
- [5] Siddhartha Devic, Charlotte Peale, Arwen Bradley, Sinead Williamson, Preetum Nakkiran, and Aravind Gollakota. Trace length is a simple uncertainty signal in reasoning models. *arXiv preprint arXiv:2510.10409*, 2025.
- [6] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. <https://transformer-circuits.pub/2021/framework/index.html>.
- [7] Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient llm post-training, 2025. URL <https://arxiv.org/abs/2510.01135>.
- [8] Evan Greensmith, Peter L. Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5:1471–1530, 2004.
- [9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng

- 373 Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X.
374 Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha,
375 Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan
376 Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu,
377 Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang,
378 and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning.
379 *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL
380 <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- 381 [10] HMMT Organization. HMMT February 2025: Harvard–MIT Mathematics Tournament. <https://www.hmmt.org/www/archive/282>, 2025. Individual round problems, February 2025, MIT,
382 Cambridge, MA.
383
- 384 [11] Baizhou Huang and Xiaojun Wan. PROS: Towards compute-efficient RLVR via rollout prefix
385 reuse. In *The Fourteenth International Conference on Learning Representations*, 2026. URL
386 <https://openreview.net/forum?id=lz1SRTcnUb>.
- 387 [12] Aaron Jaech et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- 388 [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu,
389 Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large lan-
390 guage model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium*
391 *on Operating Systems Principles*, 2023.
- 392 [14] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze
393 Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya
394 Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris
395 Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh
396 Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025.
- 397 [15] Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-
398 time intervention: Eliciting truthful answers from a language model. In A. Oh, T. Naumann,
399 A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information*
400 *Processing Systems*, volume 36, pages 41451–41530. Curran Associates, Inc., 2023. URL
401 [https://proceedings.neurips.cc/paper_files/paper/2023/file/81b8390039b](https://proceedings.neurips.cc/paper_files/paper/2023/file/81b8390039b7302c909cb769f8b6cd93-Paper-Conference.pdf)
402 [7302c909cb769f8b6cd93-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2023/file/81b8390039b7302c909cb769f8b6cd93-Paper-Conference.pdf).
- 403 [16] Zicheng Lin, Tian Liang, Jiahao Xu, Qiuzhi Lin, Xing Wang, Ruilin Luo, Chufan Shi, Siheng
404 Li, Yujiu Yang, and Zhaopeng Tu. Critical tokens matter: Token-level contrastive estimation
405 enhances llm’s reasoning capability, 2025. URL <https://arxiv.org/abs/2411.19943>.
- 406 [17] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin
407 Cai, Jeffrey Luo, Tianjun Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler:
408 Surpassing o1-preview with a 1.5b model by scaling rl. [https://pretty-radio-b75.notion](https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2)
409 [.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling](https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2)
410 [-RL-19681902c1468005bed8ca303013a4e2](https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2), 2025. Notion Blog.
- 411 [18] Mathematical Association of America. MAA American Mathematics Competitions (AMC).
412 <https://maa.org/student-programs/amc/>, 2023–2026.
- 413 [19] Mathematical Association of America. American Invitational Mathematics Examination
414 (AIME). <https://maa.org/maa-invitational-competitions/>, 2024–2026.
- 415 [20] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin,
416 Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton,
417 Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano,
418 Jan Leike, and Ryan Lowe. Training language models to follow instructions with human
419 feedback. In *Advances in Neural Information Processing Systems*, 2022.
- 420 [21] Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and
421 the geometry of large language models. In *Forty-first International Conference on Machine*
422 *Learning*, 2024. URL <https://openreview.net/forum?id=UGpGkLzwpP>.

- [22] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL <https://openreview.net/forum?id=yfYgwjj5F8>.
- [23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024. URL <https://arxiv.org/abs/2305.18290>.
- [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [25] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [26] Jeonghoon Shim, Gyuhyeon Seo, Cheongsu Lim, and Yohan Jo. Tooldial: Multi-turn dialogue generation method for tool-augmented language models. *arXiv preprint arXiv:2503.00564*, 2025.
- [27] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Na Zou, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=HvoG8SxggZ>.
- [28] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NIPS) 12*, pages 1057–1063. MIT Press, 2000. URL <https://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation>.
- [29] George Tucker, Surya Bhupatiraju, Shixiang Gu, Richard E. Turner, Zoubin Ghahramani, and Sergey Levine. The mirage of action-dependent baselines in reinforcement learning, 2018. URL <https://arxiv.org/abs/1802.10031>.
- [30] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=yfcpdY4gMP>.
- [31] Tianyi Wang, Yixia Li, Long Li, Yibiao Chen, Shaohan Huang, Yun Chen, Peng Li, Yang Liu, and Guanhua Chen. Sppo: Sequence-level ppo for long-horizon reasoning tasks, 2026. URL <https://arxiv.org/abs/2604.08865>.
- [32] Lex Weaver and Nigel Tao. The optimal reward baseline for gradient-based reinforcement learning. In *Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 538–545, 2001.
- [33] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.
- [34] Zhongwen Xu and Zihan Ding. Single-stream policy optimization, 2025. URL <https://arxiv.org/abs/2509.13232>.
- [35] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang

- 473 Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger
474 Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan
475 Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- 476 [36] Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long
477 chain-of-thought reasoning in llms, 2025. URL <https://arxiv.org/abs/2502.03373>.
- 478 [37] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai,
479 Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming
480 Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze
481 Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou,
482 Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan
483 Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- 484 [38] Huaye Zeng, Dongfu Jiang, Haozhe Wang, Ping Nie, Xiaotong Chen, and Wenhui Chen.
485 ACECODER: Acing coder RL via automated test-case synthesis. In Wanxiang Che, Joyce
486 Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the*
487 *63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long*
488 *Papers)*, pages 12023–12040, Vienna, Austria, July 2025. Association for Computational
489 Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.587. URL <https://aclanthology.org/2025.acl-long.587/>.
- 492 [39] Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. Rea-
493 soning models know when they’re right: Probing hidden states for self-verification. In *Second*
494 *Conference on Language Modeling*, 2025. URL [https://openreview.net/forum?id=06](https://openreview.net/forum?id=06I0Av7683)
495 [I0Av7683](https://openreview.net/forum?id=06I0Av7683).
- 496 [40] Yi-Kai Zhang, Yueqing Sun, Hongyan Hao, Qi Gu, Xunliang Cai, De-Chuan Zhan, and Han-
497 Jia Ye. $v_{0.5}$: Generalist value model as a prior for sparse rl rollouts, 2026. URL <https://arxiv.org/abs/2603.10848>.
- 498 [41] Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao,
499 and Beidi Chen. Act only when it pays: Efficient reinforcement learning for llm reasoning via
500 selective rollouts, 2025. URL <https://arxiv.org/abs/2506.02177>.
- 502 [42] Yubo Zhu, Dongrui Liu, Zecheng Lin, Wei Tong, Sheng Zhong, and Jing Shao. The LLM
503 already knows: Estimating LLM-perceived question difficulty via hidden representations. In
504 Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors,
505 *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*,
506 pages 1160–1176, Suzhou, China, November 2025. Association for Computational Linguistics.
507 ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.61. URL [https://aclantho](https://aclanthology.org/2025.emnlp-main.61/)
508 [logy.org/2025.emnlp-main.61/](https://aclanthology.org/2025.emnlp-main.61/).
- 509 [43] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan,
510 Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A
511 top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A Related Work

Value Estimation in RL for LLM Reasoning. RL algorithms for LLMs reasoning differ mainly in how they estimate the baseline that reduces policy-gradient variance [20, 24, 25]. Recent work extends this along two axes. The first introduces explicit value models – either a generalist value prior for sparse rollouts [40] or a sequence-level value model that treats reasoning as a contextual bandit [31] – but incurs the substantial training, calibration, and deployment cost of an additional LLM-scale model. The second reduces rollout cost by non-uniform prompt sampling, via probabilistic informativeness-based filtering [41] or historical value tracking with global advantage normalization [34]; however, both rely on initial rollouts or per-prompt reward histories, which become prohibitive when rollout generation dominates RLVR cost [11]. In contrast, our method predicts the baseline at negligible cost by reusing the policy’s hidden states and generation signals already computed during the forward pass, eliminating both an auxiliary value model and pre-collected rollouts while preserving the variance-reduction benefit of value estimation.

Outcome-relevant Information in Hidden States. A growing body of work shows that the hidden states of large language models encode information relevant to assessing their outputs and task outcomes. Early studies on language model interpretability support this by showing that simple linear probes over internal representations can recover latent properties such as factuality [3], truthfulness [1, 15], confidence [43], and answer correctness. Recent studies extend this idea to reasoning models, where activations have been used to predict final correctness [4], identify capability boundaries between solvable and unsolvable prompts [42], estimate perceived difficulty and reasoning effort [39], and support self-verification or early stopping during generation [27].

Overall, prior work primarily uses hidden-state information as diagnostic signals or test-time control mechanisms. In contrast, we incorporate such signals directly into RL training by learning an online value estimator from the policy model’s own hidden states, yielding a cheap baseline without requiring an auxiliary LLM-scale critic or large rollout groups.

537 **B Theoretical Proofs**

538 **B.1 Proof of Proposition 1**

539 *Proof.* Define:

$$\mu(x) = \mathbb{E}[Z(x, y) \mid x], \quad \Sigma_w = \mathbb{E}_x[\text{Cov}(Z(x, y) \mid x)], \quad \Sigma_b = \text{Cov}_x(\mu(x)).$$

540 Let $G_i = \frac{1}{m} \sum_{j=1}^m Z(x_i, G_{ij})$. Since completions within a prompt are conditionally independent
541 given x_i ,

$$\text{Cov}(G_i \mid x_i) = \frac{1}{m} \text{Cov}(Z(x_i, y) \mid x_i).$$

542 Applying the law of total covariance to G_i :

$$\text{Cov}(G_i) = \mathbb{E}_x[\text{Cov}(G_i \mid x_i)] + \text{Cov}_x(\mathbb{E}[G_i \mid x_i]) = \frac{1}{m} \Sigma_w + \Sigma_b.$$

543 Since prompts are sampled independently, $\text{Cov}(\hat{g}) = \frac{1}{n} \text{Cov}(G_i)$. Substituting $n = B/m$:

$$\text{Cov}(\hat{g}) = \frac{m}{B} \left(\frac{1}{m} \Sigma_w + \Sigma_b \right) = \frac{1}{B} \Sigma_w + \frac{m}{B} \Sigma_b. \quad \square$$

544 **B.2 Proof of Corollary 1**

545 *Proof.* From Proposition 1, for any allocation (n, m) with $nm = B$,

$$\text{Cov}(\hat{g}) = \frac{1}{B} \Sigma_w + \frac{m}{B} \Sigma_b.$$

546 Consider any two allocations $m_1 < m_2$ with the same budget B . Their difference is

$$\text{Cov}(\hat{g}_{m_2}) - \text{Cov}(\hat{g}_{m_1}) = \frac{m_2 - m_1}{B} \Sigma_b.$$

547 Since $\Sigma_b = \text{Cov}_x(\mu(x))$ is a covariance matrix, it is positive semidefinite, so for any vector v ,

$$v^\top (\text{Cov}(\hat{g}_{m_2}) - \text{Cov}(\hat{g}_{m_1})) v = \frac{m_2 - m_1}{B} v^\top \Sigma_b v \geq 0.$$

548 Therefore $\text{Cov}(\hat{g}_{m_2}) \succeq \text{Cov}(\hat{g}_{m_1})$ for any $m_2 > m_1$, meaning the variance of $\hat{g}$ in every gradient
549 direction is non-decreasing in m . The minimum is thus attained at $m = 1$, giving $n = B$. $\square$

550 C Implementation Details

551 C.1 Pseudocode for POISE

Algorithm 1 POISE

Require: Prompt distribution $\mathcal{D}$; initial policy π_{θ_0} ; initial value estimator g_{f_0} ; prompt batch size M ; value-buffer size n ; PPO clip ϵ .

- 1: Initialize $\theta \leftarrow \theta_0$, $f \leftarrow f_0$, value buffer $\mathcal{B}_V \leftarrow \emptyset$.
- 2: **for** step = 1, 2, ..., T **do**
- 3: $\theta_{\text{old}} \leftarrow \theta$
- 4: Sample a mini-batch of prompts $\{x_b\}_{b=1}^M \sim \mathcal{D}$.
- 5: $\mathcal{R} \leftarrow \emptyset$, $\mathcal{S}_V \leftarrow \emptyset$
- 6: **for** each prompt x_b **do**
- 7: Sample $y_b^{(1)}, y_b^{(2)} \stackrel{\text{i.i.d.}}{\sim} \pi_{\theta_{\text{old}}}(\cdot | x_b)$.
- 8: Extract internal state features $\phi_b^{(i)} \leftarrow \phi_{\theta_{\text{old}}}(x_b, y_b^{(i)})$ for $i \in \{1, 2\}$ via forward hooks.
- 9: Compute rewards $r_b^{(i)} \leftarrow R(x_b, y_b^{(i)})$ for $i \in \{1, 2\}$.
- 10: Compute cross-rollout baselines:
 $b_b^{(1)} \leftarrow g_f(\phi_b^{(2)}), \quad b_b^{(2)} \leftarrow g_f(\phi_b^{(1)})$.
- 11: Compute advantages $A_b^{(i)} \leftarrow r_b^{(i)} - b_b^{(i)}$ for $i \in \{1, 2\}$.
- 12: Add policy-update examples:
 $\mathcal{R} \leftarrow \mathcal{R} \cup \{(x_b, y_b^{(i)}, A_b^{(i)})\}_{i=1,2}$.
- 13: Add value-estimator examples:
 $\mathcal{S}_V \leftarrow \mathcal{S}_V \cup \{(\phi_b^{(1)}, r_b^{(2)}), (\phi_b^{(2)}, r_b^{(1)})\}$.
- 14: **end for**
- 15: Update θ by maximizing the PPO objective in Eq. (14) on $\mathcal{R}$.
- 16: Update f by minimizing

$$\sum_{(\phi, \tilde{r}) \in \mathcal{B}_V \cup \mathcal{S}_V} (g_f(\phi) - \tilde{r})^2.$$
- 17: Append $\mathcal{S}_V$ to $\mathcal{B}_V$; evict examples older than n steps.
- 18: **end for**
- 19: **return** π_θ, g_f .

552 C.2 Baseline Algorithm

553 We adopt DAPO [37] as our baseline RL algorithm. For each prompt $q \sim \mathcal{D}$, a group of G responses
 554 $\{o^{(i)}\}_{i=1}^G$ is sampled from the old policy $\pi_{\theta_{\text{old}}}$, and the following objective is optimized:

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q, a) \sim \mathcal{D}, \{o^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)} \left[\frac{1}{\sum_{i=1}^G |o^{(i)}|} \sum_{i=1}^G \sum_{t=1}^{|o^{(i)}|} \min \left(r_t^{(i)}(\theta) \hat{A}_t^{(i)}, \text{clip}(r_t^{(i)}(\theta), 1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}) \hat{A}_t^{(i)} \right) \right], \quad (15)$$

$$\text{s.t. } 0 < |\{o^{(i)} \mid \text{is_equivalent}(a, o^{(i)})\}| < G,$$

556 where the importance ratio and the group-relative advantage are

$$r_t^{(i)}(\theta) = \frac{\pi_\theta(o_t^{(i)} | q, o_{<t}^{(i)})}{\pi_{\theta_{\text{old}}}(o_t^{(i)} | q, o_{<t}^{(i)})}, \quad \hat{A}_t^{(i)} = \frac{R^{(i)} - \text{mean}(\{R^{(j)}\}_{j=1}^G)}{\text{std}(\{R^{(j)}\}_{j=1}^G)}.$$

557 DAPO augments GRPO with four key techniques: (i) **Clip-Higher** decouples the lower and upper
 558 clipping bounds ($\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$), giving low-probability tokens more room to be promoted and
 559 mitigating entropy collapse; (ii) **Dynamic Sampling** filters out prompts whose responses are all
 560 correct or all incorrect, ensuring that every batch yields effective gradients; (iii) **Token-Level Policy**
 561 **Gradient Loss** normalizes by the total token count $\sum_i |o^{(i)}|$ rather than per-sequence, so longer

562 responses contribute proportionally to the loss; (iv) **Overlong Reward Shaping** applies a length-
 563 aware penalty to truncated samples to reduce reward noise.

564 **Reward.** Unlike the original DAPO, we drop the length penalty and use a purely binary reward
 565 based solely on correctness for simplicity:

$$R^{(i)} = \begin{cases} 1, & \text{if } o^{(i)} \text{ is correct,} \\ 0, & \text{otherwise.} \end{cases}$$

566 C.3 Value Estimator

567 Hidden states used for estimator training are collected from the model’s teacher-forced log probability
 568 forward pass, which is already required by our policy optimization algorithm and therefore does
 569 not need additional computation. Specifically, the implementation registers a forward hook on
 570 one transformer layer during this log probability computation, and pools prompt hidden states and
 571 reasoning hidden states separately.

572 Table 4 lists the hyperparameters used by the hidden-state value estimator. The prompt and reasoning
 573 hidden-state features are obtained by mean-pooling the last 10 prompt tokens and the last 10 reasoning
 574 tokens, respectively. For a pair of rollouts $(y^{(1)}, y^{(2)})$ from the same prompt, the supervised target
 575 for the feature row of $y^{(i)}$ is the paired rollout reward $\tilde{R}^{(i)} = R(x, y^{(j)}), j \neq i$.

Component	Setting
Hidden layer	Qwen3-4B: 19 DeepSeek-R1-Distill-Qwen-1.5B: 19
Prompt hidden state pooling	Last 10 prompt token mean
Reasoning hidden state pooling	Last 10 reasoning token mean
Scalar features	3 entropy statistics
Final estimator input dimension	$2d_{\text{model}} + 3$ Qwen3-4B: 5123 DeepSeek-R1-Distill-Qwen-1.5B: 3075
Regressor	StandardScaler + Ridge
Ridge penalty	$\alpha = 0.01$
Random seed	42
Prediction range	Clipped to $[0, 1]$

Table 4: Hyperparameters for the hidden-state value estimator. Here, d_{model} denotes the hidden-state dimension of each policy backbone.

576 C.4 Training details

577 We instantiate our method on two reasoning model backbones, Qwen3-4B and DeepSeek-R1-Distill-
 578 Qwen-1.5B, with all training implemented in the `verl` library and the maximum response length
 579 capped at 8,192 tokens. As training data, we use DAPO-Math-17K [37] after filtering out Chinese-
 580 language prompts, yielding an English-only mathematical reasoning corpus. We use a training batch
 581 size of 1,024 prompts for Qwen3-4B and 512 prompts for DeepSeek-R1-Distill-Qwen-1.5B, and
 582 sample rollouts with temperature 1.0 and top-p 1.0. All experiments are run on B200 GPUs. Our
 583 main baseline is DAPO [37], for which we adopt the implementation and hyperparameters of Zheng
 584 et al. [41], which improves the efficiency of DAPO’s dynamic sampling. DAPO baseline draws 8
 585 rollouts per prompt while our method draws a pair of rollouts per prompt and forms its baseline
 586 through the cross-rollout value probe described in § 3. Detailed hyperparameter values are reported
 587 in Table 5 and 6.

588 C.5 Evaluation Protocol

589 The following section details the benchmarks used in our experiments and our evaluation protocol.

590 Benchmarks.

Hyperparameter	Value
algorithm	POISE
training steps	100
train batch size	512
mini batch size	16
max prompt length	2048
max response length	8192
learning rate	1×10^{-6}
clip ratio (low / high)	0.2 / 0.28
entropy coefficient	0
use KL loss	False
sampling temperature	1.0
sampling top-p	1.0
samples per prompt	2
max batched tokens	10240

Table 5: Key training hyperparameters used in POISE.

Hyperparameter	Value
algorithm	DAPO
training steps	100
train batch size	128
mini batch size	16
p_{easy}	0.5
p_{hard}	0.5
target zero variance	0.25
default br size	192
GRESO min p	0.05
GRESO max p	0.95
β	1.25
max prompt length	2048
max response length	8192
learning rate	1×10^{-6}
clip ratio (low / high)	0.2 / 0.28
entropy coefficient	0
use KL loss	False
sampling temperature	1.0
sampling top-p	1.0
samples per prompt	8
max batched tokens	10240

Table 6: Key training hyperparameters used in DAPO with efficient dynamic sampling [41].

- **AMC23** and **AMC24** are problem sets from the American Mathematics Competition (AMC) [18], a series of contests for high school students that test problem-solving ability across a wide range of topics. We employ the 2023 and 2024 editions, consisting of 40 problems each (80 problems in total).
- **AIME24**, **AIME25**, and **AIME26** are problem sets from the American Invitational Mathematics Examination (AIME) [19], a prestigious competition featuring challenging problems that require sophisticated mathematical reasoning. We employ the 2024, 2025, and 2026 editions, consisting of 30 problems each (90 problems in total).
- **HMMT25** consists of problems from the February 2025 Harvard–MIT Mathematics Tournament (HMMT) [10], one of the most demanding high-school mathematics competitions in the United States. We use the individual-round problems, consisting of 30 problems.

602 • **BRUMO25** is the 2025 edition of the Brown University Mathematics Olympiad
 603 (BRUMO) [2], an annual olympiad-level competition for advanced high-school students.
 604 We use the official 2025 problem set, consisting of 30 problems.

605 **Evaluation Protocol** For all benchmarks above, we follow the officially recommended decoding
 606 setting with temperature 0.6 and top- p 0.95, and set the maximum response length to 8192 tokens the
 607 same as the training setting. All inferences are performed with the vLLM library [13] on a single
 608 node equipped with two NVIDIA B200 GPUs. For each problem, we independently sample 32
 609 completions.

610 We define a binary reward $r^{(ij)} \in \{0, 1\}$ equal to 1 if the j -th response to problem i yields a correct
 611 final answer and 0 otherwise; the same reward function is used during RL training and evaluation.
 612 Given a test set with M problems, we report:

613 • **avg@ k** : the expected per-sample correctness,

$$\text{avg@}k = \frac{1}{M} \sum_{i=1}^M \frac{1}{k} \sum_{j=1}^k r^{(ij)}.$$

614 Throughout the paper we use $k = 32$ for both metrics.

615 C.6 Prompt Template

616 We use the following single-turn prompt template, following DAPO [37], for all mathematical
 617 reasoning tasks. The placeholder {problem} is replaced with the problem statement at training and
 618 inference time.

Prompt Template

Solve the following math problem step by step. The last line of your response should be of the form
 Answer: \$Answer (without quotes) where \$Answer is the answer to the problem.

{problem}

Remember to put your answer on its own line after “Answer:”.

619

620 D Training Dynamics

621 We provide additional training dynamics for POISE on Qwen3-4B and DeepSeek-R1-Distill-Qwen-
 622 1.5B. In the main text, we report aggregate evidence that the internal state value estimator remains
 623 stable during training. Here, we expand the analysis by tracking five quantities over policy updates:
 624 the batch reward, the predicted value, token-level entropy, the value-estimation error against the
 625 online target, and the advantage variance ratio.

626 For each training step t , let R_t denote the verifier reward of the sampled rollouts, $b_t = g_f(\phi_t)$
 627 the predicted baseline, and $A_t = R_t - b_t$ the resulting advantage. We report the batch reward
 628 $\bar{R}_t = \mathbb{E}_{\text{batch}}[R_t]$, the mean predicted value $\bar{b}_t = \mathbb{E}_{\text{batch}}[b_t]$, the online target error $\text{MAE}_t =$
 629 $\mathbb{E}_{\text{batch}}[|b_t - \hat{V}_t(x)|]$, and the advantage variance ratio $\rho_t = \text{Var}_{\text{batch}}(A_t)/\text{Var}_{\text{batch}}(R_t)$. Here, $\hat{V}_t(x)$
 630 is the empirical online target estimated from rollouts sampled from the current checkpoint policy. The
 631 ratio ρ_t measures how much variance remains after subtracting the learned baseline; values below
 632 one indicate that the estimator reduces variance relative to using the raw reward.

633 **Qwen3-4B.** Figure 5 shows the full dynamics for Qwen3-4B. The batch reward increases rapidly
 634 during the first phase of training, from roughly 0.35 to above 0.65, and then stabilizes around 0.70.
 635 The predicted value follows the same broad trend, rising from approximately 0.30 to the 0.70-0.75
 636 range. This agreement suggests that the online estimator tracks the changing reward scale as the
 637 policy improves, rather than remaining calibrated only to the initial policy.

638 The online target MAE decreases sharply in early training and then remains in a stable range. This is
 639 important because the value function itself is nonstationary during RL: as the actor improves, the
 640 target expected reward for each prompt also changes. The stability of the MAE therefore indicates
 641 that the sliding-buffer update is sufficient for tracking the evolving policy. The advantage variance
 642 ratio remains below one for most of training and stabilizes after the initial phase, showing that the
 643 learned baseline continues to reduce variance after the policy has moved away from its initialization.
 644 Finally, entropy increases rather than collapsing, suggesting that POISE does not obtain its gains by
 645 prematurely concentrating the policy distribution.



Figure 4: The green line reports the online $\text{MAE}(g_f(\phi), \bar{R}_t)$, where $\bar{R}_t$ is the mean reward of the rollouts at step t . The red line reports the variance reduction ratio, $1 - \text{Var}(A)/\text{Var}(R)$, where $A = R - b$ is the advantage.

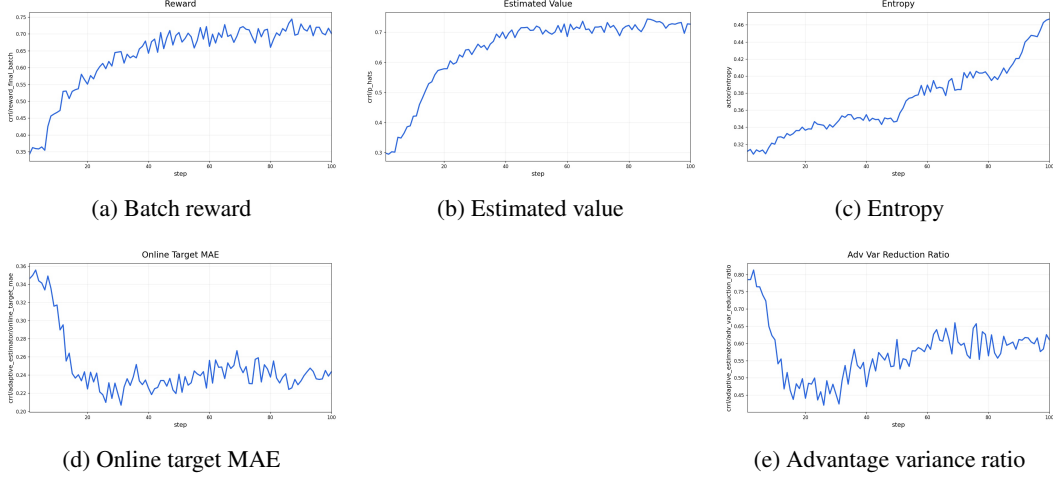


Figure 5: Training dynamics of POISE on Qwen3-4B. The reward and predicted value increase together as the policy improves. The online target MAE remains stable after the early phase, and the advantage variance ratio stays below one for most of training, indicating that the learned baseline reduces reward variance when forming advantages.

646 **DeepSeek-R1-Distill-Qwen-1.5B.** Figure 6 reports the same diagnostics for DeepSeek-R1-Distill-
 647 Qwen-1.5B. The smaller backbone shows noisier dynamics, which is expected because its reward
 648 distribution is lower and more variable. Even in this setting, the reward improves throughout training,
 649 increasing from roughly 0.15-0.20 in the initial phase to around 0.45-0.50 by the end of training. The
 650 estimated value also increases over the course of training, although it exhibits an early drop before
 651 recovering. This transient mismatch likely reflects the difficulty of fitting the online estimator in the
 652 first few updates, when the buffer contains limited data and the policy distribution changes quickly.

653 After this initial phase, the estimator becomes more stable. The online target MAE remains bounded
 654 and does not diverge as training proceeds, indicating that the estimator continues to track the current
 655 policy despite the nonstationary target. The advantage variance ratio also falls below one after the
 656 early updates and remains there for most of the run, showing that the learned baseline provides
 657 variance reduction even for the smaller and noisier policy. Entropy rises during the early-to-middle
 658 phase and then fluctuates mildly, suggesting that POISE maintains policy stochasticity rather than
 659 driving immediate entropy collapse.

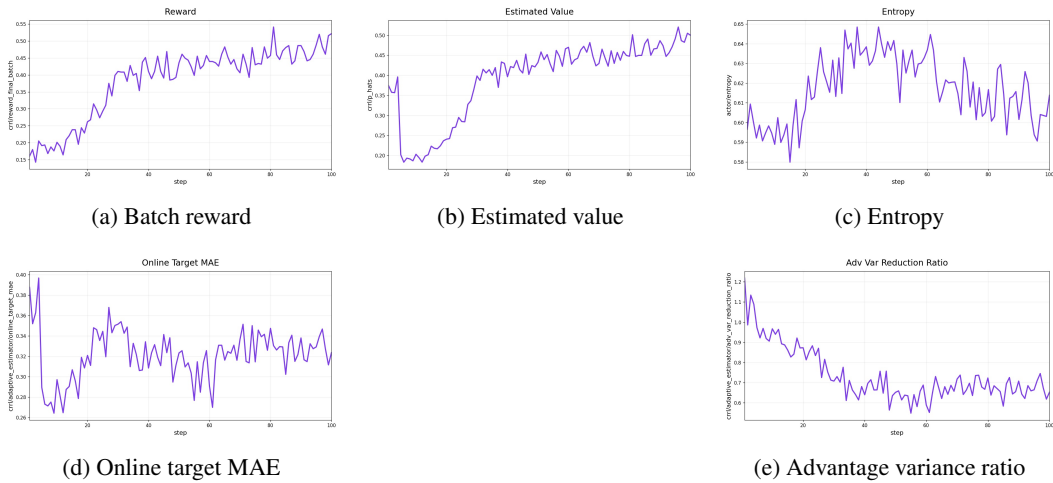


Figure 6: Training dynamics of POISE on DeepSeek-R1-Distill-Qwen-1.5B. Although the smaller model exhibits noisier dynamics, the reward improves steadily and the estimated value follows the increasing reward scale after the initial phase. The online target MAE remains bounded, and the advantage variance ratio stays below one for most of training, indicating effective variance reduction.

E Comparison to Policy-model Scale Critic Model

E.1 Critic Model Implementation

For the critic baseline in the value-prediction comparison, we implement a sequence-level scalar critic following SPPO [31]. Unlike the token-level PPO critic, which predicts a value $V(s_t)$ for every intermediate token state and relies on GAE to propagate sparse terminal rewards, the SPPO critic treats long-form reasoning as a sequence-level contextual bandit. The prompt x is the context, the full response y is the action, and the verifier reward $R(x, y)$ is an outcome-level binary reward. Accordingly, the critic predicts a single prompt-level scalar value

$$V_\phi(x) \approx \mathbb{E}_{y \sim \pi(\cdot|x)}[R(x, y)], \quad (16)$$

which can be interpreted as the policy’s estimated probability of solving the prompt.

Architecturally, the critic is initialized from the corresponding policy backbone and augmented with a scalar value head. We feed the chat-formatted prompt into the model, collect the hidden state at the final non-padding token, and apply a linear head to produce a scalar logit:

$$z_\phi(x) = w^\top h_\phi(x), \quad V_\phi(x) = \sigma(z_\phi(x)). \quad (17)$$

In the POISE comparison, the critic is trained only as an analysis baseline and is never used to compute POISE advantages. Given reward-labeled rollouts for a prompt, we aggregate independently sampled verifier rewards into an empirical prompt value, e.g. Avg@ K :

$$\hat{V}(x) = \frac{1}{K} \sum_{j=1}^K R(x, y^{(j)}). \quad (18)$$

The critic is then optimized to predict this prompt-level target. Specifically, we use the Bernoulli objective,

$$\mathcal{L}_{\text{BCE}}(\phi) = \text{BCEWithLogitsLoss}(z_\phi(x), R). \quad (19)$$

At evaluation time, we compare critic predictions against held-out empirical prompt values using mean absolute error (MAE) and Pearson correlation.

E.2 Comparison with an Online Policy-model Scale Critic

§ 3.1 evaluated value prediction on a fixed policy distribution. We further test whether the internal state estimator remains competitive in the online setting, where the policy changes over the course of RL training. At every 10 training steps, we take the corresponding actor checkpoint, sample eight responses for each prompt used in the step, and use the empirical Avg@8 score as the target prompt value. We then compare two estimators against this target: a separately trained policy-scale critic and our lightweight internal state estimator.

Figures 7 and 8 show the result up to step 100 for Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B. We report both Pearson correlation and mean absolute error (MAE) to the Avg@8 target.

For both Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B, the internal state estimator closely tracks the critic throughout training. While a separate critic can be slightly more accurate because it is an LLM-scale model trained on accumulated rollout data, our estimator remains close to the critic across both model scales while using only hidden-state and entropy features already produced by the policy forward pass. This supports the central motivation of POISE: internal states provide a practical value signal that tracks the evolving policy without maintaining an additional critic model.

E.3 Comparisons on Multiple Domains and Models

We evaluate whether our internal-state value estimator generalizes to different verifiable-reward domains and policy backbones. We consider five datasets spanning math reasoning, coding, tool use, and instruction following: DAPO-Math [37], DeepScaleR [17], AceCoder [38], ToolDial [26], and IF-RLVR [22]. We also evaluate three policy backbones: Qwen3-4B, DeepSeek-R1-Distill-Qwen-1.5B, and DeepSeek-R1-Distill-Qwen-7B.

For each model–dataset pair, we train the internal-state estimator and a separately trained critic on the same reward-labeled rollout data. To make the comparison consistent across settings, we use 4,096

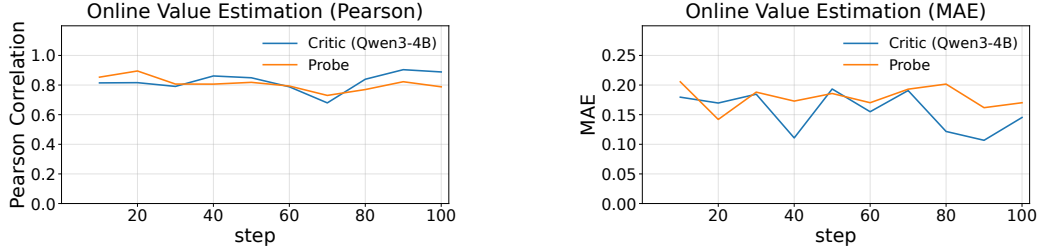


Figure 7: Online value prediction for Qwen3-4B. The target at each checkpoint is the empirical Avg@8 score from the corresponding actor checkpoint.

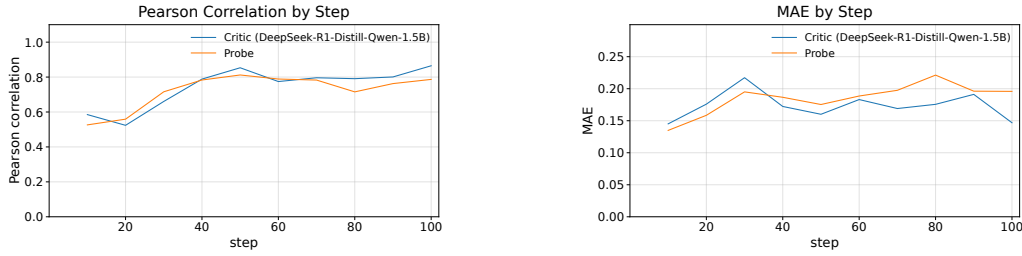


Figure 8: Online value prediction for DeepSeek-R1-Distill-Qwen-1.5B. The target at each checkpoint is the empirical Avg@8 score from the corresponding actor checkpoint.

702 training examples for each estimator, matching the number of examples stored in the value-estimator
 703 buffer used by the POISE algorithm. We then evaluate both estimators against empirical Avg@8
 704 values on held-out prompts. For evaluation, we use 1,024 held-out prompts for all datasets except
 705 ToolDial, for which we use 13,492 held-out examples.

706 Table 7 reports the results. On Qwen3-4B, the internal state estimator consistently outperforms the
 707 critic across all five domains, reducing MAE and improving Pearson correlation in every setting.
 708 The gains are especially large outside the original math setting: on AceCoder, Pearson correlation
 709 increases from 0.056 to 0.612, and on IF-RLVR it increases from 0.150 to 0.642. These results
 710 indicate that the estimator is not merely exploiting dataset-specific artifacts from DAPO-Math; it can
 711 recover value-relevant information from policy activations across substantially different forms of
 712 verifiable feedback.

713 The trend is more mixed but still favorable on DeepSeek-R1-Distill-Qwen-1.5B. The internal state
 714 estimator improves over the critic on DeepScaleR and ToolDial, while the critic is stronger on
 715 AceCoder and slightly better in MAE on IF-RLVR. Even in these weaker cases, the estimator remains
 716 competitive in correlation despite using only lightweight hidden-state and entropy features. For the
 717 completed DeepSeek-R1-Distill-Qwen-7B settings, the estimator again improves over the critic on
 718 both DAPO-Math and ToolDial, suggesting that the signal persists at a larger model scale.

719 Overall, these results support the broader applicability of POISE beyond the mathematical-reasoning
 720 training experiments.

Table 7: Full value-prediction results across policy backbones and verifiable-reward domains. We compare our internal state estimator against a separately trained critic and report MAE and Pearson correlation r .

Policy model	Domain	Dataset	Critic		internal state estimator	
			MAE ↓	r ↑	MAE ↓	r ↑
Qwen3-4B	Math reasoning	DAPO-Math	0.262	0.676	0.141	0.870
		DeepScaleR	0.393	0.384	0.231	0.609
	Coding	AceCoder	0.499	0.056	0.234	0.612
	Tool calling	ToolDial	0.303	0.440	0.188	0.840
	Instruction following	IF-RLVR	0.350	0.150	0.195	0.642
DeepSeek-R1-Distill-Qwen-1.5B	Math reasoning	DAPO-Math	0.127	0.723	0.123	0.834
		DeepScaleR	0.251	0.586	0.151	0.829
	Coding	AceCoder	0.135	0.706	0.196	0.580
	Tool calling	ToolDial	0.201	0.337	0.122	0.672
	Instruction following	IF-RLVR	0.101	0.545	0.171	0.557
DeepSeek-R1-Distill-Qwen-7B	Math reasoning	DAPO-Math	0.300	0.441	0.191	0.784
		DeepScaleR	0.270	0.457	0.164	0.814
	Coding	AceCoder	0.229	0.691	0.173	0.804
	Tool calling	ToolDial	0.194	0.663	0.153	0.842
	Instruction following	IF-RLVR	0.155	0.716	0.191	0.596

F Ablations

F.1 Ablations of Hyperparameters During Hidden State Extraction

We next ablate the hidden-state extraction hyperparameters of the Qwen3-4B estimator. Unless otherwise specified, we keep the estimator architecture fixed as StandardScaler followed by ridge regression, with prompt hidden states projected to 32 dimensions and trajectory hidden states projected to 256 dimensions using PCA. The scalar entropy features are kept fixed across all runs.

Layer index. We first fix the pooling window to the last 10 tokens and sweep the transformer layer used to extract both prompt and trajectory hidden states. For Qwen3-4B, Table 8 shows that performance is already strong in early layers but improves toward the middle of the network. The best Pearson correlation is obtained at layer 19, while layer 33 gives the lowest MAE. We use layer 19 as the default for the main experiments because it gives the strongest correlation with verifier value while remaining near-optimal in MAE.

Table 8: Layer ablation for Qwen3-4B with last-10-token mean pooling.

Layer	MAE ↓	Pearson r ↑
1	0.128	0.807
3	0.126	0.821
5	0.129	0.816
7	0.129	0.820
9	0.131	0.814
11	0.131	0.819
13	0.130	0.823
15	0.128	0.828
17	0.124	0.831
19	0.123	0.834
21	0.128	0.830
23	0.130	0.824
25	0.132	0.817
27	0.125	0.827
29	0.125	0.830
31	0.126	0.824
33	0.120	0.831
35	0.123	0.827

For DeepSeek-R1-Distill-Qwen-1.5B, we repeat the same odd-layer sweep using prompt hidden states and reasoning-trajectory hidden states, both mean-pooled over the last 10 tokens. As shown in Table 9, the strongest validation performance appears in the earliest layer. However, the differences across layers are small in absolute MAE, and layer 19 remains close to the best setting while using the same extraction configuration as Qwen3-4B. We therefore use layer 19 for both backbones in the main POISE experiments to avoid model-specific layer tuning and keep the estimator implementation consistent across models.

Pooling window. We also vary the number of final tokens used for mean pooling while fixing the layer to 19. As shown in Table 10, last-10 and last-15 pooling perform almost identically, while last-5 pooling is weaker. We choose last-10 pooling as the default because it achieves the best Pearson correlation while using a shorter extraction window than last-15.

Table 10: Pooling-window ablation for Qwen3-4B at layer 19.

Pooling window	MAE ↓	Pearson r ↑
Last 5	0.127	0.822
Last 10	0.124	0.834
Last 15	0.124	0.833

Table 9: Layer ablation for DeepSeek-R1-Distill-Qwen-1.5B with last-10-token mean pooling.

Layer	MAE ↓	Pearson r ↑
1	0.134	0.425
3	0.135	0.405
5	0.135	0.418
7	0.136	0.403
9	0.137	0.369
11	0.141	0.332
13	0.139	0.339
15	0.137	0.380
17	0.136	0.382
19	0.135	0.395
21	0.135	0.378
23	0.135	0.332
25	0.139	0.290
27	0.139	0.292

For DeepSeek-R1-Distill-Qwen-1.5B, we fix the layer to 1, which was selected by the layer sweep, and repeat the pooling-window ablation in Table 11. The three pooling windows are close, but last-10 pooling gives the best MAE and Pearson correlation.

Table 11: Pooling-window ablation for DeepSeek-R1-Distill-Qwen-1.5B at layer 1.

Pooling window	MAE ↓	Pearson r ↑
Last 5	0.1342	0.4230
Last 10	0.1339	0.4247
Last 15	0.1342	0.4200

Overall, these ablations support the robustness of the hidden-state extraction choice used in the main POISE experiments. The estimator is not overly sensitive to the exact pooling window, and while the best layer can be model-dependent, a shared layer-19 configuration provides competitive performance for both backbones. We therefore use layer 19 with last-10-token mean pooling as the default extraction setting in the main experiments.

F.2 Ablations on Probe Designs

We ablate the architecture of the lightweight value estimator while keeping the input representation fixed. All models use the Qwen3-4B features from layer 19 with last-10-token mean pooling, including prompt hidden states, trajectory hidden states, and entropy statistics. The train/test split are identical to those used in § F.1. We compare the default linear ridge regressor against multi-layer perceptrons (MLPs) with different depths and widths. MLPs are trained with AdamW, ReLU activations, dropout, and early stopping on a held-out validation split from the training set.

Table 12 shows that increasing model capacity can slightly reduce MAE: the best MLP, a 3-layer network with width 1024, improves MAE from 0.124 to 0.117. However, this improvement does not translate into better rank alignment with the target values. The linear ridge probe achieves the highest Pearson correlation, 0.834, while all MLP variants obtain lower correlation. We therefore use the linear estimator in the main method: it is cheaper to fit online, has fewer hyperparameters, and provides the most reliable correlation with verifier value, which is important for forming stable advantages.

Table 12: Probe architecture ablation for Qwen3-4B at layer 19 with last-10-token mean pooling. MLP names indicate depth $\times$ hidden width. Lower MAE and higher Pearson correlation are better.

Probe architecture	MAE $\downarrow$	Pearson r $\uparrow$
Linear ridge	0.124	0.834
MLP 1 $\times$ 512	0.154	0.778
MLP 3 $\times$ 512	0.123	0.780
MLP 5 $\times$ 512	0.128	0.765
MLP 7 $\times$ 512	0.124	0.763
MLP 9 $\times$ 512	0.133	0.814
MLP 3 $\times$ 1024	0.117	0.801
MLP 5 $\times$ 1024	0.128	0.754

Overall, this result suggests that the value-relevant signal in the policy’s internal states is largely linearly accessible. Larger nonlinear probes can improve absolute calibration in some cases, but they are less stable in correlation and introduce additional online-training cost. This supports our choice of a linear probe as the default estimator for POISE.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction (§1) state three main claims: (i) POISE matches DAPO on mathematical reasoning while requiring less compute, (ii) the internal-state value estimator surpasses a separately trained policy-scale critic, and (iii) the estimator generalizes to other verifiable-reward domains. These are supported respectively by §4.2 (Table 1, Figure 3), §3.1 and §5 (Figure 1, Table 2), and §5 (Table 2 / Table 7).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: §7 (“Limitations and Future Work”) explicitly discusses that experiments are conducted under a fixed compute budget, that behavior under substantially longer training horizons is not characterized, and that the estimator currently provides only sequence-level (not token-level) value, leaving finer credit assignment to future work.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The paper presents Proposition 1 (gradient variance decomposition) and Corollary 1 (optimal allocation) in §2.3 with all required assumptions stated, and the unbiasedness condition for baselines in §2.2 (Eqs. 6–7). Full proofs are provided in §B.1 and §B.2 of the appendix, and theorems are properly numbered and cross-referenced.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: §4.1 describes models, datasets, decoding settings, hardware, and the baseline implementation; §3.2 and Algorithm 1 (§C.1) specify the full POISE procedure; §C.3–C.6 list value-estimator hyperparameters (Table 4), POISE/DAPO training hyperparameters (Tables 5–6), evaluation protocol, and the prompt template used at training and inference.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We will release an anonymized implementation of POISE (built on the verl library) together with training and evaluation scripts as supplemental material. All datasets used (DAPO-Math-17K, DeepScaleR, AceCoder, ToolDial, IF-RLVR) and base models (Qwen3-4B, DeepSeek-R1-Distill-Qwen-1.5B/7B) are publicly available, and §C–E describe the exact configuration needed to reproduce results.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: §4.1 reports the core training and evaluation setup (models, datasets, batch sizes, decoding, hardware), and §C.3–C.5 in the appendix provide full details on data preprocessing, optimizer settings, clip ratios, learning rate, value-estimator hyperparameters (Table 4), and the per-method hyperparameter tables for POISE (Table 5) and DAPO (Table 6). The hidden-state extraction hyperparameters and probe architecture choices are further ablated in §E.3.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: To control the substantial compute cost of RLVR training, our main reasoning-benchmark results (Table 1) report Avg@32 across 32 independent samples per problem, which already provides a stable Monte Carlo estimate of expected pass rate, but we do not report seed-level error bars across full RL training runs. Training-dynamics curves (Figures 3, 6, 7) and value-estimation analyses (Figures 1, 4, 5, 8, 9; Tables 2, 7) are reported as point estimates without confidence intervals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: §4.1 and §4.2 specify that all training is run on NVIDIA B200 GPUs and report wall-clock times to reach matched performance: about 18 hours (POISE) vs. 24 hours (DAPO) on a single B200 for DeepSeek-R1-Distill-Qwen-1.5B, and about 37 hours (POISE) vs. 49 hours (DAPO) on two B200s for Qwen3-4B. §C.5 also notes that evaluation uses two NVIDIA B200 GPUs with vLLM.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: We have reviewed the NeurIPS Code of Ethics. The work uses publicly available reasoning benchmarks and open-weight models, does not involve human subjects or personally identifying data, and the submission is anonymized.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: POISE is a generic, foundational change to the value-estimation step of RLVR for reasoning models; it improves compute efficiency and training stability but is not tied to a specific deployment, dataset, or downstream application. As a methodological contribution that does not introduce new data or model capabilities beyond those of standard RLVR baselines, we do not foresee direct societal impacts beyond those already shared with RLHF/RLVR research broadly.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: We do not release new pre-trained models, image generators, or scraped datasets. Our released artifacts consist solely of training/evaluation code and configuration for an RL algorithm operating on publicly available open-weight reasoning models and existing public benchmarks, which do not pose elevated misuse risk relative to the underlying assets.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All base models (Qwen3-4B [35], DeepSeek-R1-Distill-Qwen-1.5B/7B [9]), datasets (DAPO-Math-17K [37], DeepScaleR [17], AceCoder [38], ToolDial [26], IF-RLVR [22]), and infrastructure (verl, vLLM [13], the GRESO/DAPO implementation of [41]) are cited at the points where they are used (§4.1, §C.3–C.5, §E.2), and we use them under their respective licenses for non-commercial research.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new asset is the POISE training code (algorithm, internal-state value estimator, and configuration files). It will be released as anonymized supplemental material with a README documenting installation, training command, hyperparameters (matching Tables 4–6), and evaluation scripts.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.

- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects; all experiments are run on publicly available models and benchmarks.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve research with human subjects, so no IRB approval is required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs (Qwen3-4B and DeepSeek-R1-Distill-Qwen-1.5B/7B) are the policy models being trained and analyzed by our RL algorithm, which is the standard subject of study in RLVR research rather than a non-standard methodological component. We did not use LLMs as an original or non-standard part of the methodology beyond their role as the trained policy.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.

1141
1142

- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.